



UNIVERSIDAD ESTATAL AMAZÓNICA

Unidad de Organización Curricular: **Básica**

Campo de Estudio: **Tronco común**

INFORME DE PRÁCTICAS DEL COMPONENTE PRÁCTICO-EXPERIMENTAL	
Nombre(s) del(de los) Estudiante(s): Jean Carlos Delgado Cepeda	
Asignatura: Estructura de datos	Calificación:
Fecha del informe: 21/06/2026	
N° Práctica: 01	
Título de la Práctica: Identificación de tipos de datos	
Introducción	1. Introducción La presente práctica experimental desarrolla una solución informática para administrar turnos de pacientes en una clínica. El caso fue seleccionado porque representa una situación frecuente en instituciones de salud pequeñas y medianas: el registro manual de pacientes, la asignación de citas y la consulta de información pueden generar duplicidad de datos, pérdida de tiempo y errores de coordinación. Desde el punto de vista técnico, la solución se construye con Programación Orientada a Objetos (POO), porque este paradigma permite representar entidades del problema mediante clases y objetos. La documentación oficial de Python indica que Python admite clases y objetos, y que las listas son estructuras capaces de almacenar varios elementos en una misma colección (Python Software Foundation, 2026a; Python Software Foundation, 2026b). La práctica también aplica estructuras de datos estudiadas hasta la semana 4: vectores, matrices, registros y estructuras. En Python, estas estructuras se implementan mediante listas, listas anidadas y clases de datos. Esta decisión permite mantener el código comprensible, modular y adecuado para un proyecto académico inicial.
	1.1 Fundamentación teórica La Programación Orientada a Objetos organiza el programa alrededor de objetos que poseen atributos y comportamientos. En el sistema propuesto, un paciente y un turno se modelan como objetos; sus atributos almacenan datos como cédula, nombres, fecha, hora, especialidad, médico y estado. La documentación de IBM describe la POO como una forma de modelar el problema objetivo dentro de los programas,

favoreciendo la reutilización y la disminución de errores (IBM, s. f.).

Una estructura de datos permite organizar información para consultarla, modificarla y recorrerla de manera ordenada. En esta práctica se usan vectores para almacenar pacientes y turnos; una matriz para representar horarios disponibles; registros para agrupar los datos de cada paciente y turno; y estructuras de control para validar las operaciones del sistema.

1.2 Antecedentes

En una clínica, los turnos pueden gestionarse de forma manual mediante cuadernos, hojas de cálculo o llamadas telefónicas. Aunque esos mecanismos pueden funcionar en escenarios pequeños, presentan limitaciones: dificultad para consultar citas anteriores, riesgo de asignar dos turnos en el mismo horario, ausencia de reportería y poca trazabilidad. Por ello, se propone un sistema básico de agenda clínica que centraliza la información en memoria y demuestra el uso de POO y estructuras de datos.

1.3 Justificación

El sistema se justifica porque permite registrar pacientes, asignar turnos, consultar información y generar reportes por especialidad. A nivel académico, el proyecto evidencia la relación entre el análisis de un problema real y su implementación mediante programación. A nivel práctico, la solución ayuda a comprender cómo un sistema pequeño puede crecer hacia una aplicación más completa con base de datos, interfaz gráfica y autenticación de usuarios.

2. Análisis de la problemática

Caso seleccionado: agenda de turnos de pacientes de una clínica.

Elemento	Descripción
Necesidad identificada	Controlar el registro de pacientes y la asignación de turnos médicos evitando duplicidad de horarios y mejorando la consulta de información.
Problema principal	El registro manual dificulta encontrar pacientes, revisar turnos, cancelar citas y obtener reportes por especialidad.
Objetivo del sistema	Diseñar una aplicación de consola que registre, consulte y visualice pacientes y turnos aplicando POO y estructuras de datos.

Alcance	Sistema académico ejecutado en consola, con almacenamiento temporal en memoria y datos de demostración.
---------	---

2.1 Usuarios involucrados

Usuario	Rol dentro del sistema
Recepcionista	Registra pacientes, agenda turnos, consulta información y cancela turnos.
Paciente	Solicita un turno y proporciona sus datos personales básicos.
Médico	Recibe la agenda organizada por fecha, hora y especialidad.
Administrador académico del sistema	Revisa el código, la estructura de datos y las evidencias de funcionamiento.

2.2 Procesos principales

Proceso	Entrada	Salida esperada
Registrar paciente	Cédula, nombres, edad, teléfono	Paciente almacenado en el vector de pacientes.
Registrar turno	Cédula, fecha, hora, especialidad, médico	Turno almacenado si el paciente existe y el horario no está ocupado.
Consultar paciente	Cédula	Datos del paciente y turnos asociados.
Visualizar turnos	Opción del menú	Listado completo de turnos registrados.
Generar reporte	Opción del menú	Conteo de turnos activos por especialidad.
Cancelar turno	Código del turno	Cambio de estado del turno a "Cancelado".

	2.3 Datos gestionados <table border="1"> <thead> <tr> <th>Entidad</th><th>Datos</th></tr> </thead> <tbody> <tr> <td>Paciente</td><td>cedula, nombres, edad, telefono</td></tr> <tr> <td>Turno</td><td>codigo, cedula_paciente, fecha, hora, especialidad, medico, estado</td></tr> <tr> <td>Disponibilidad</td><td>matriz de horarios dividida en jornada matutina y vespertina</td></tr> <tr> <td>Especialidades</td><td>vector con Medicina general, Pediatría, Odontología y Ginecología</td></tr> </tbody> </table>	Entidad	Datos	Paciente	cedula, nombres, edad, telefono	Turno	codigo, cedula_paciente, fecha, hora, especialidad, medico, estado	Disponibilidad	matriz de horarios dividida en jornada matutina y vespertina	Especialidades	vector con Medicina general, Pediatría, Odontología y Ginecología
Entidad	Datos										
Paciente	cedula, nombres, edad, telefono										
Turno	codigo, cedula_paciente, fecha, hora, especialidad, medico, estado										
Disponibilidad	matriz de horarios dividida en jornada matutina y vespertina										
Especialidades	vector con Medicina general, Pediatría, Odontología y Ginecología										
Desarrollo o procedimiento	3. Desarrollo de la solución informática La solución fue implementada en Python mediante una aplicación de consola. Se eligió este lenguaje porque permite trabajar con clases, listas, listas anidadas y estructuras de datos de manera clara. El programa se organiza en tres partes: definición de registros mediante clases de datos, clase principal de control AgendaClinica y menú interactivo para el usuario. 3.1 Escenario de desarrollo <table border="1"> <thead> <tr> <th>Aspecto</th><th>Descripción</th></tr> </thead> <tbody> <tr> <td>Lugar de trabajo</td><td>Escritorio de estudio con computador personal.</td></tr> <tr> <td>Sistema operativo</td><td>Windows 11 o distribución Linux compatible con Python 3.</td></tr> <tr> <td>Herramientas</td><td>Python 3, editor de código, terminal o símbolo del sistema, Git y GitHub.</td></tr> <tr> <td>Tipo de práctica</td><td>Aplicación experimental de consola con evidencias de ejecución.</td></tr> </tbody> </table> 3.2 Procedimiento aplicado 1. Identificación del problema y selección del caso de estudio. 2. Definición de usuarios, procesos y datos principales. 3. Diseño de clases Paciente, Turno y AgendaClinica. 4. Implementación de vectores para pacientes y turnos.	Aspecto	Descripción	Lugar de trabajo	Escritorio de estudio con computador personal.	Sistema operativo	Windows 11 o distribución Linux compatible con Python 3.	Herramientas	Python 3, editor de código, terminal o símbolo del sistema, Git y GitHub.	Tipo de práctica	Aplicación experimental de consola con evidencias de ejecución.
Aspecto	Descripción										
Lugar de trabajo	Escritorio de estudio con computador personal.										
Sistema operativo	Windows 11 o distribución Linux compatible con Python 3.										
Herramientas	Python 3, editor de código, terminal o símbolo del sistema, Git y GitHub.										
Tipo de práctica	Aplicación experimental de consola con evidencias de ejecución.										

5. Implementación de matriz de disponibilidad de horarios.
6. Programación de funciones de registro, consulta, visualización, reporte y cancelación.
7. Ejecución del sistema con datos de prueba.
8. Captura de evidencias y elaboración del informe técnico.

3.3 Diseño orientado a objetos

Clase	Responsabilidad	Atributos o métodos principales
Paciente	Representar los datos básicos de una persona atendida por la clínica.	cedula, nombres, edad, telefono
Turno	Representar una cita médica asignada a un paciente.	codigo, cedula_paciente, fecha, hora, especialidad, medico, estado
AgendaClinica	Gestionar pacientes, turnos, validaciones y reportes.	registrar_paciente, buscar_paciente, registrar_turno, visualizar_turnos, reporte_por_especialidad

3.4 Explicación de la solución implementada

El sistema inicia creando un objeto de la clase AgendaClinica. Luego carga datos de demostración para facilitar la prueba inicial. El menú permite seleccionar operaciones. Cuando se registra un paciente, el sistema valida que no exista otra persona con la misma cédula. Cuando se registra un turno, se comprueba que el paciente exista, que la especialidad sea válida y que el médico no tenga otro turno en la misma fecha y hora.

La reportería se implementa mediante un recorrido del vector de turnos. Por cada turno activo, se incrementa el contador de su especialidad. Este procedimiento permite visualizar el volumen de citas por área médica y demuestra el uso de estructuras de datos para obtener información resumida.

4. Análisis de las estructuras de datos utilizadas

Estructura	Implementación	Uso en el sistema	Justificación
Vector	Lista de pacientes	self.pacientes	Permite almacenar múltiples

				registros de pacientes y recorrerlos para búsqueda secuencial.
	Vector	Lista de turnos	self.turnos	Permite registrar y visualizar todas las citas médicas creadas.
	Matriz	Lista anidada	self.matriz_disponibilidad	Representa horarios por jornada, facilitando la idea de filas y columnas.
	Registro	dataclass Paciente	Paciente(...)	Agrupar datos relacionados de una misma entidad.
	Registro	dataclass Turno	Turno(...)	Organiza la información de cada cita médica en una estructura clara.
	Estructura	Clase AgendaClinica	Objeto controlador	Integra datos y operaciones bajo un diseño orientado a objetos.

4.1 Ventajas

Las listas permiten agregar nuevos datos con facilidad, recorrer los elementos mediante ciclos y mantener un código simple. Las clases de datos facilitan la lectura del programa porque cada entidad posee atributos definidos. La matriz ayuda a representar horarios por bloques, lo cual permite relacionar la práctica con el concepto de filas y columnas.

4.2 Limitaciones

La información se almacena solo en memoria, por lo que se pierde al cerrar el programa. La búsqueda de pacientes y turnos es secuencial; por tanto, si el número de registros crece demasiado, el sistema puede volverse menos eficiente. Además, la aplicación no incluye interfaz gráfica

	ni conexión con base de datos. 4.3 Posibles mejoras Como mejora futura, el sistema podría incorporar una base de datos SQLite o MySQL, una interfaz gráfica con Tkinter, validación avanzada de fechas, exportación de reportes a PDF, autenticación de usuarios y sincronización con un repositorio GitHub real.															
Resultados	5. Resultados Durante la ejecución se comprobó que el sistema permite registrar pacientes, listar turnos, consultar por cédula, cancelar citas y generar reportes por especialidad. Los datos de demostración permiten evidenciar que la solución funciona sin depender de una base de datos externa.															
	<table><tr><th>Prueba</th><th>Acción realizada</th><th>Resultado obtenido</th></tr><tr><td>Prueba 1</td><td>Visualizar todos los turnos cargados</td><td>El sistema muestra tres turnos iniciales con paciente, fecha, hora, especialidad, médico y estado.</td></tr><tr><td>Prueba 2</td><td>Generar reporte por especialidad</td><td>El sistema cuenta un turno en Medicina general, uno en Pediatría y uno en Odontología.</td></tr><tr><td>Prueba 3</td><td>Consultar paciente por cédula</td><td>El sistema muestra los datos del paciente y sus turnos asociados.</td></tr><tr><td>Prueba 4</td><td>Cancelar turno</td><td>El sistema cambia el estado de un turno a Cancelado.</td></tr></table>	Prueba	Acción realizada	Resultado obtenido	Prueba 1	Visualizar todos los turnos cargados	El sistema muestra tres turnos iniciales con paciente, fecha, hora, especialidad, médico y estado.	Prueba 2	Generar reporte por especialidad	El sistema cuenta un turno en Medicina general, uno en Pediatría y uno en Odontología.	Prueba 3	Consultar paciente por cédula	El sistema muestra los datos del paciente y sus turnos asociados.	Prueba 4	Cancelar turno	El sistema cambia el estado de un turno a Cancelado.
	Prueba	Acción realizada	Resultado obtenido													
	Prueba 1	Visualizar todos los turnos cargados	El sistema muestra tres turnos iniciales con paciente, fecha, hora, especialidad, médico y estado.													
	Prueba 2	Generar reporte por especialidad	El sistema cuenta un turno en Medicina general, uno en Pediatría y uno en Odontología.													
Prueba 3	Consultar paciente por cédula	El sistema muestra los datos del paciente y sus turnos asociados.														
Prueba 4	Cancelar turno	El sistema cambia el estado de un turno a Cancelado.														

Figura 1. Evidencia de ejecución del menú y operaciones principales.

Evidencia: ejecución del sistema agenda_clinica.py

```
SISTEMA DE AGENDA DE TURNOS - CLINICA
1. Registrar paciente
2. Registrar turno
3. Consultar paciente por cedula
4. Visualizar todos los turnos
5. Reporte por especialidad
6. Cancelar turno
7. Salir
Seleccione una opcion:
LISTADO DE TURNOS
1. Ana Maria Lopez | 2026-06-24 | 08:00 | Medicina general | Dra. Rivera | Agendado
2. Carlos Perez Zambrano | 2026-06-24 | 09:00 | Odontologia | Dr. Molina | Agendado
3. Lucia Torres Vega | 2026-06-25 | 10:00 | Pediatria | Dra. Cedeño | Agendado

SISTEMA DE AGENDA DE TURNOS - CLINICA
1. Registrar paciente
2. Registrar turno
3. Consultar paciente por cedula
4. Visualizar todos los turnos
5. Reporte por especialidad
6. Cancelar turno
7. Salir
Seleccione una opcion:
REPORTE POR ESPECIALIDAD
Medicina general: 1 turno(s)
Pediatria: 1 turno(s)
Odontologia: 1 turno(s)
Ginecologia: 0 turno(s)

SISTEMA DE AGENDA DE TURNOS - CLINICA
1. Registrar paciente
2. Registrar turno
3. Consultar paciente por cedula
```

Figura 2. Reporte por especialidad generado a partir del vector de turnos.

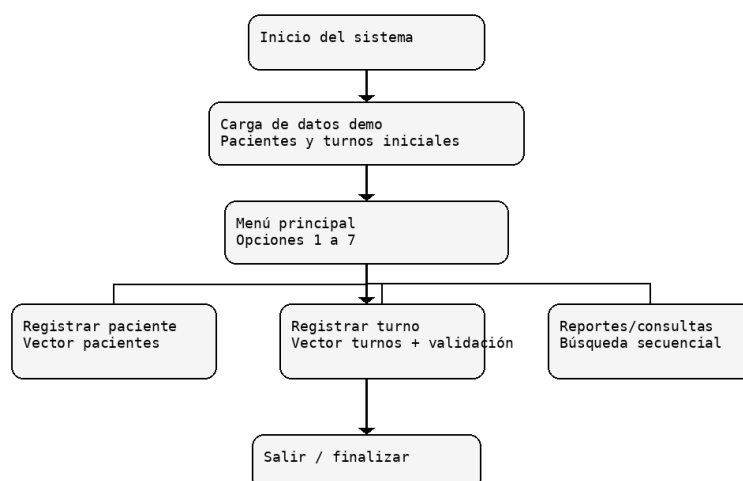
Reporte por especialidad

Especialidad	Turnos
Medicina general	1
Pediatria	1
Odontologia	1
Ginecologia	0

Resultado esperado: el sistema cuenta los turnos activos por cada especialidad definida en el vector especialidades.

Figura 3. Diagrama general del funcionamiento del sistema.

Diagrama general del funcionamiento del sistema



Conclusiones	6. Conclusiones • La práctica permitió aplicar la Programación Orientada a Objetos en un problema concreto relacionado con la gestión de turnos clínicos. • El uso de vectores, matrices, registros y estructuras facilitó la organización de los datos y permitió demostrar operaciones de registro, consulta, visualización y reportería. • La solución desarrollada cumple el objetivo académico porque evidencia la relación entre el análisis de una necesidad real y su implementación en código. • La principal limitación del sistema es que almacena los datos en memoria; por ello, una mejora importante sería integrar una base de datos y una interfaz gráfica.
Bibliografía	7. Bibliografía American Psychological Association. (s. f.). APA Style. https://apastyle.apa.org/ IBM. (s. f.). Programación orientada a objetos. IBM Documentation. https://www.ibm.com/docs/es/spss-modeler/saas?topic=language-object-oriented-programming Python Software Foundation. (2026a). Data structures. Python 3.14.6 documentation. https://docs.python.org/3/tutorial/datastructures.html Python Software Foundation. (2026b). Data model. Python 3.14.6 documentation. https://docs.python.org/3/reference/datamodel.html Python Software Foundation. (2026c). dataclasses — Data classes. Python 3.14.6 documentation. https://docs.python.org/3/library/dataclasses.html
Anexos	8. Anexos Anexo A. Código fuente completo <pre> from dataclasses import dataclass from datetime import datetime from typing import List, Optional @dataclass class Paciente: cedula: str nombres: str edad: int telefono: str @dataclass class Turno: codigo: int cedula_paciente: str fecha: str hora: str especialidad: str medico: str estado: str = "Agendado" </pre>

```

class AgendaClinica:
    def __init__(self):
        self.pacientes: List[Paciente] = [] # vector de registros
        self.turnos: List[Turno] = [] # vector de registros
        self.matriz_disponibilidad = [ # matriz: dias x
horarios
            ["08:00", "09:00", "10:00", "11:00"],
            ["14:00", "15:00", "16:00", "17:00"]
        ]
        self.especialidades = ["Medicina general", "Pediatria",
"Odontologia", "Ginecologia"]

    def registrar_paciente(self, paciente: Paciente) -> bool:
        if self.buscar_paciente(paciente.cedula) is not None:
            return False
        self.pacientes.append(paciente)
        return True

    def buscar_paciente(self, cedula: str) -> Optional[Paciente]:
        for paciente in self.pacientes:
            if paciente.cedula == cedula:
                return paciente
        return None

    def registrar_turno(self, cedula: str, fecha: str, hora: str,
especialidad: str, medico: str) -> bool:
        if self.buscar_paciente(cedula) is None:
            return False
        if especialidad not in self.especialidades:
            return False
        if self.existe_turno(fecha, hora, medico):
            return False
        codigo = len(self.turnos) + 1
        self.turnos.append(Turno(codigo, cedula, fecha, hora,
especialidad, medico))
        return True

    def existe_turno(self, fecha: str, hora: str, medico: str) ->
bool:
        for turno in self.turnos:
            if turno.fecha == fecha and turno.hora == hora and
turno.medico.lower() == medico.lower() and turno.estado !=
"Cancelado":
                return True
        return False

    def consultar_turnos_por_cedula(self, cedula: str) ->
List[Turno]:
        return [turno for turno in self.turnos if
turno.cedula_paciente == cedula]

    def visualizar_turnos(self) -> List[Turno]:
        return self.turnos

    def reporte_por_especialidad(self):
        reporte = {}
        for especialidad in self.especialidades:
            reporte[especialidad] = 0
        for turno in self.turnos:
            if turno.estado != "Cancelado":
                reporte[turno.especialidad] += 1
        return reporte

    def cancelar_turno(self, codigo: int) -> bool:
        for turno in self.turnos:
            if turno.codigo == codigo:
                turno.estado = "Cancelado"
                return True

```

	<pre> return False def cargar_datos_demo(agenda: AgendaClinica): agenda.registrar_paciente(Paciente("1723456789", "Ana Maria Lopez", 28, "0991112223")) agenda.registrar_paciente(Paciente("0912345678", "Carlos Perez Zambrano", 41, "0983334445")) agenda.registrar_paciente(Paciente("2309876543", "Lucia Torres Vega", 10, "0975556667")) agenda.registrar_turno("1723456789", "2026-06-24", "08:00", "Medicina general", "Dra. Rivera") agenda.registrar_turno("0912345678", "2026-06-24", "09:00", "Odontologia", "Dr. Molina") agenda.registrar_turno("2309876543", "2026-06-25", "10:00", "Pediatria", "Dra. Cedeño") def mostrar_menu(): print("\nSISTEMA DE AGENDA DE TURNOS - CLINICA") print("1. Registrar paciente") print("2. Registrar turno") print("3. Consultar paciente por cedula") print("4. Visualizar todos los turnos") print("5. Reporte por especialidad") print("6. Cancelar turno") print("7. Salir") def main(): agenda = AgendaClinica() cargar_datos_demo(agenda) while True: mostrar_menu() opcion = input("Seleccione una opcion: ") if opcion == "1": cedula = input("Cedula: ") nombres = input("Nombres: ") edad = int(input("Edad: ")) telefono = input("Telefono: ") ok = agenda.registrar_paciente(Paciente(cedula, nombres, edad, telefono)) print("Paciente registrado correctamente." if ok else "Error: ya existe un paciente con esa cedula.") elif opcion == "2": cedula = input("Cedula del paciente: ") fecha = input("Fecha AAAA-MM-DD: ") hora = input("Hora HH:MM: ") especialidad = input("Especialidad: ") medico = input("Medico: ") ok = agenda.registrar_turno(cedula, fecha, hora, especialidad, medico) print("Turno registrado correctamente." if ok else "Error: paciente inexistente, especialidad invalida o turno ocupado.") elif opcion == "3": cedula = input("Cedula a consultar: ") paciente = agenda.buscar_paciente(cedula) if paciente: print(f"Paciente: {paciente.nombres} Edad: {paciente.edad} Telefono: {paciente.telefono}") for turno in agenda.consultar_turnos_por_cedula(cedula): print(f"Turno {turno.codigo}: {turno.fecha} {turno.hora} - {turno.especialidad} - {turno.medico} - {turno.estado}") else: print("No se encontro el paciente.") elif opcion == "4": print("\nLISTADO DE TURNOS") </pre>
--	--

```

        for turno in agenda.visualizar_turnos():
            paciente =
agenda.buscar_paciente(turno.cedula_paciente)
            nombre = paciente.nombres if paciente else "Sin
datos"
            print(f"{turno.codigo}. {nombre} | {turno.fecha} |
{turno.hora} | {turno.especialidad} | {turno.medico} |
{turno.estado}")
            elif opcion == "5":
                print("\nREPORTE POR ESPECIALIDAD")
                for especialidad, total in
agenda.reporte_por_especialidad().items():
                    print(f"{especialidad}: {total} turno(s)")
            elif opcion == "6":
                codigo = int(input("Codigo de turno a cancelar: "))
                print("Turno cancelado." if agenda.cancelar_turno(codigo)
else "No existe el turno.")
            elif opcion == "7":
                print("Gracias por usar el sistema.")
                break
            else:
                print("Opcion invalida.")

if __name__ == "__main__":
    main()

```

Anexo B. Enlace al repositorio GitHub

No puedo confirmar un enlace real de GitHub porque no tengo acceso para crear repositorios en la cuenta del estudiante. Debe reemplazarse por el enlace definitivo, por ejemplo: <https://github.com/usuario/agenda-turnos-clinica>

Anexo C. Agente de IA utilizado

Agente utilizado: ChatGPT. Porcentaje estimado de apoyo: 60 %. El estudiante revisó, adaptó y ejecutó el código, mientras que el agente colaboró en la redacción técnica, estructuración del informe y propuesta inicial del código. Este porcentaje debe ajustarse si el estudiante modifica más secciones antes de la entrega.



Jean Carlos Delgado Cepeda